489 Case

In [1]:
```
import os
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

#statsmodels
import statsmodels.api as sm

#Scikit
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import precision_score, confusion_matrix, accuracy_score
from sklearn.model_selection import cross_val_score, train_test_split, cross_val_predict

#torch
import torch
import torch.nn as nn
import torch.utils.data as Data
from torch.autograd import Variable

#captum
from captum.attr import IntegratedGradients
from captum.attr import LayerConductance
from captum.attr import NeuronConductance

def func_turn_df_into_t_float(df_arg):
    t_ret = torch.from_numpy(np.array(df_arg))
    t_ret = t_ret.to(torch.float)
    return t_ret

def func_turn_df_into_t_long(df_arg):
    t_ret = torch.from_numpy(np.array(df_arg))
    t_ret = t_ret.to(torch.long)
    return t_ret

mySEED = 2023
myRANDOM_STATE = mySEED # for split train/eval
myTEST_PERCENTAGE = 0.1
torch.manual_seed(mySEED) #always first
```

```
targetDirectory = "C:\\CASES\\PYTPRA"
os.chdir(targetDirectory)#hier sucht Python nach csv Datei
data = pd.read_csv('CaseBGB489.csv', delimiter=';', decimal = ',') # STROKE
# Überprüfen auf redundante und/oder NaN Daten
if data.duplicated().sum() > 0:
    data = data.drop_duplicates()
    print('Redundante Daten wurden entfernt.')
    print('-------------------------')

if data.isna().any().sum() > 0:
    data = data.dropna()
    print('NaN-Daten wurden entfernt.')
    print('-------------------------')
print(data.shape)
print(data.head())
print("Working directory in use: ", os. getcwd())
```

```
Redundante Daten wurden entfernt.
-------------------------
(138, 9)
   AUSUEBUNG  SELBSTAENDIG  BERATUNGSOFFEN  ALTER  MONATSNETTO  NOMINALZINS  \
0        1.0           0.0             0.0   61.0          0.0         5.39
1        0.0           0.0             1.0   46.0     150000.0         4.30
2        0.0           0.0             1.0   54.0      10000.0         5.08
3        0.0           0.0             0.0   62.0          0.0         5.00
4        1.0           0.0             1.0   58.0      90000.0         4.95

   GESAMTZUSAGE  URSPRUNGSLAUFZEIT  RECHTVERFUEGBARSEIT
0       90000.0              180.0                 23.0
1       70000.0              180.0                  1.0
2      120000.0              181.0                 54.0
3       70000.0              166.0                 29.0
4      460000.0              179.0                 15.0
Working directory in use:  C:\CASES\PYTPRA
```

In [2]:

```
X = data[['SELBSTAENDIG', 'BERATUNGSOFFEN', 'ALTER', 'MONATSNETTO', 'NOMINALZINS', 'GESAMTZUSAGE',
          'URSPRUNGSLAUFZEIT', 'RECHTVERFUEGBARSEIT']]
y = data['AUSUEBUNG']


# Datensatz in Trainings- und Testdaten aufteilen

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=myTEST_PERCENTAGE, random_state=myRANDOM_STATE)
```

```
x_temp = sm.add_constant(X_train)
log_reg = sm.Logit(y_train, x_temp).fit()
print(log_reg.summary())
```

```
Optimization terminated successfully.
         Current function value: 0.313769
         Iterations 8
                           Logit Regression Results
==============================================================================
Dep. Variable:              AUSUEBUNG   No. Observations:                  124
Model:                          Logit   Df Residuals:                      115
Method:                           MLE   Df Model:                            8
Date:                Thu, 13 Feb 2025   Pseudo R-squ.:                  0.4012
Time:                        14:26:59   Log-Likelihood:                -38.907
converged:                       True   LL-Null:                       -64.980
Covariance Type:            nonrobust   LLR p-value:                 1.577e-08
============================================================================================
                               coef    std err          z      P>|z|      [0.025      0.975]
--------------------------------------------------------------------------------------------
const                   -19.8710      5.519     -3.601      0.000     -30.687      -9.055
SELBSTAENDIG              1.5052      0.723      2.081      0.037       0.087       2.923
BERATUNGSOFFEN            1.4906      0.686      2.173      0.030       0.146       2.835
ALTER                    -0.0350      0.028     -1.242      0.214      -0.090       0.020
MONATSNETTO          -1.212e-05    5.6e-06     -2.163      0.031   -2.31e-05   -1.14e-06
NOMINALZINS               1.5677      0.705      2.224      0.026       0.186       2.949
GESAMTZUSAGE         -9.179e-07   8.98e-07     -1.022      0.307   -2.68e-06    8.42e-07
URSPRUNGSLAUFZEIT         0.0794      0.019      4.173      0.000       0.042       0.117
RECHTVERFUEGBARSEIT      -0.0639      0.021     -3.115      0.002      -0.104      -0.024
============================================================================================
```

In [3]:
```
# Standardisierung
scaler = StandardScaler()
scaler.fit(X_train)
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

In [4]:
```
#prep torch data
t_X_train = func_turn_df_into_t_float(X_train_scaled)
t_X_valid = func_turn_df_into_t_float(X_test_scaled)

t_y_train = func_turn_df_into_t_long(y_train)
t_y_valid = func_turn_df_into_t_long(y_test)
```

In [5]:
```
myHIDDENone = 6 #we have 8 dim input + 1 out = 9, two thirds = 6
myMAX_EPOCHS = 500 #500 #more than 2000 does not pay off, valid error does not change thereafter, 500 is ok
myLEARNING_RATE = 0.1
myBATCH_SIZE = 32 #32 is min
```

In [6]:
```
#PyTorch MLP

torch_dataset = Data.TensorDataset(t_X_train, t_y_train)
train_loader = Data.DataLoader(
    dataset=torch_dataset,
    batch_size=myBATCH_SIZE,
    shuffle=True, num_workers=0,)#use default

model_pt = nn.Sequential(nn.Linear(8, myHIDDENone),
                      nn.Sigmoid(),
                      nn.Linear(myHIDDENone, 2), nn.Softmax(dim=1))
```

In [7]:
```
loss_fn = nn.CrossEntropyLoss() #watch this!
optimizer =  torch.optim.Adam(model_pt.parameters(), lr=myLEARNING_RATE)
#torch.optim.SGD(model_pt.parameters(), lr=0.5) is worse
losses_train = []
losses_val = []

def generic_training_loop_extended(n_epochs, optimizer, model, loss_fn, train_t_u, val_t_u,train_t_c, val_t_c):
    for epoch in range(1, n_epochs + 1):
        loss_train_avg = []
        for step, (batch_x, batch_y) in enumerate(train_loader):
            b_x = Variable(batch_x)
            b_y = Variable(batch_y)
            train_t_p = model(b_x) #all details are in model
            train_loss = loss_fn(train_t_p, b_y) #and the loss function
            loss_train_avg.append(train_loss.item())
            optimizer.zero_grad()
            train_loss.backward()
            optimizer.step()
        with torch.no_grad():
            val_t_p = model(val_t_u)
            val_loss = loss_fn(val_t_p, val_t_c)
            losses_val.append(val_loss.item())
            assert val_loss.requires_grad == False
        losses_train.append(train_loss.item())
        if epoch <= 100 or epoch % 50 == 0:
```

```
            print("Epoch", {epoch}, f"Avg. training loss over all batches: {np.average(loss_train_avg[0:step]):.4f}",
                  f"Validation loss:  {val_loss.item():.4f}",
                  "\n \tLast batch out of", {step}, f"has training loss: {train_loss.item():.4f}")

    return

generic_training_loop_extended(n_epochs = myMAX_EPOCHS, optimizer = optimizer, model = model_pt,
                       loss_fn = loss_fn , train_t_u = t_X_train, val_t_u = t_X_valid,
                       train_t_c = t_y_train, val_t_c = t_y_valid)
```

```
Epoch {1} Avg. training loss over all batches: 0.6161 Validation loss:  0.5975
        Last batch out of {3} has training loss: 0.5467
Epoch {2} Avg. training loss over all batches: 0.5222 Validation loss:  0.5956
        Last batch out of {3} has training loss: 0.5184
Epoch {3} Avg. training loss over all batches: 0.5462 Validation loss:  0.5941
        Last batch out of {3} has training loss: 0.4145
Epoch {4} Avg. training loss over all batches: 0.4910 Validation loss:  0.5893
        Last batch out of {3} has training loss: 0.5657
Epoch {5} Avg. training loss over all batches: 0.4775 Validation loss:  0.5771
        Last batch out of {3} has training loss: 0.5563
Epoch {6} Avg. training loss over all batches: 0.4816 Validation loss:  0.5597
        Last batch out of {3} has training loss: 0.4812
Epoch {7} Avg. training loss over all batches: 0.4502 Validation loss:  0.5408
        Last batch out of {3} has training loss: 0.5194
Epoch {8} Avg. training loss over all batches: 0.4700 Validation loss:  0.5285
        Last batch out of {3} has training loss: 0.4196
Epoch {9} Avg. training loss over all batches: 0.4607 Validation loss:  0.5295
        Last batch out of {3} has training loss: 0.4116
Epoch {10} Avg. training loss over all batches: 0.4537 Validation loss:  0.5383
        Last batch out of {3} has training loss: 0.4093
Epoch {11} Avg. training loss over all batches: 0.4453 Validation loss:  0.5536
        Last batch out of {3} has training loss: 0.4145
Epoch {12} Avg. training loss over all batches: 0.4331 Validation loss:  0.5501
        Last batch out of {3} has training loss: 0.4284
Epoch {13} Avg. training loss over all batches: 0.4072 Validation loss:  0.5457
        Last batch out of {3} has training loss: 0.4970
Epoch {14} Avg. training loss over all batches: 0.4326 Validation loss:  0.5308
        Last batch out of {3} has training loss: 0.3888
Epoch {15} Avg. training loss over all batches: 0.4380 Validation loss:  0.5186
        Last batch out of {3} has training loss: 0.3584
Epoch {16} Avg. training loss over all batches: 0.4029 Validation loss:  0.5136
        Last batch out of {3} has training loss: 0.4703
Epoch {17} Avg. training loss over all batches: 0.4106 Validation loss:  0.4992
        Last batch out of {3} has training loss: 0.4279
Epoch {18} Avg. training loss over all batches: 0.4089 Validation loss:  0.4779
        Last batch out of {3} has training loss: 0.4235
Epoch {19} Avg. training loss over all batches: 0.4215 Validation loss:  0.4624
        Last batch out of {3} has training loss: 0.3689
Epoch {20} Avg. training loss over all batches: 0.4092 Validation loss:  0.4510
        Last batch out of {3} has training loss: 0.4029
Epoch {21} Avg. training loss over all batches: 0.4047 Validation loss:  0.4473
        Last batch out of {3} has training loss: 0.4107
Epoch {22} Avg. training loss over all batches: 0.3918 Validation loss:  0.4463
        Last batch out of {3} has training loss: 0.4412
```

```
Epoch {23} Avg. training loss over all batches: 0.4023 Validation loss:  0.4445
        Last batch out of {3} has training loss: 0.3967
Epoch {24} Avg. training loss over all batches: 0.4028 Validation loss:  0.4427
        Last batch out of {3} has training loss: 0.3927
Epoch {25} Avg. training loss over all batches: 0.3874 Validation loss:  0.4442
        Last batch out of {3} has training loss: 0.4358
Epoch {26} Avg. training loss over all batches: 0.4065 Validation loss:  0.4514
        Last batch out of {3} has training loss: 0.3602
Epoch {27} Avg. training loss over all batches: 0.4048 Validation loss:  0.4636
        Last batch out of {3} has training loss: 0.3620
Epoch {28} Avg. training loss over all batches: 0.4040 Validation loss:  0.4681
        Last batch out of {3} has training loss: 0.3571
Epoch {29} Avg. training loss over all batches: 0.3886 Validation loss:  0.4748
        Last batch out of {3} has training loss: 0.4043
Epoch {30} Avg. training loss over all batches: 0.3905 Validation loss:  0.4729
        Last batch out of {3} has training loss: 0.3900
Epoch {31} Avg. training loss over all batches: 0.3752 Validation loss:  0.4773
        Last batch out of {3} has training loss: 0.4334
Epoch {32} Avg. training loss over all batches: 0.3922 Validation loss:  0.4720
        Last batch out of {3} has training loss: 0.3660
Epoch {33} Avg. training loss over all batches: 0.3889 Validation loss:  0.4715
        Last batch out of {3} has training loss: 0.3753
Epoch {34} Avg. training loss over all batches: 0.3725 Validation loss:  0.4701
        Last batch out of {3} has training loss: 0.4134
Epoch {35} Avg. training loss over all batches: 0.3919 Validation loss:  0.4659
        Last batch out of {3} has training loss: 0.3377
Epoch {36} Avg. training loss over all batches: 0.3931 Validation loss:  0.4609
        Last batch out of {3} has training loss: 0.3272
Epoch {37} Avg. training loss over all batches: 0.3764 Validation loss:  0.4547
        Last batch out of {3} has training loss: 0.3752
Epoch {38} Avg. training loss over all batches: 0.3667 Validation loss:  0.4486
        Last batch out of {3} has training loss: 0.4009
Epoch {39} Avg. training loss over all batches: 0.3693 Validation loss:  0.4421
        Last batch out of {3} has training loss: 0.3874
Epoch {40} Avg. training loss over all batches: 0.3862 Validation loss:  0.4372
        Last batch out of {3} has training loss: 0.3269
Epoch {41} Avg. training loss over all batches: 0.3643 Validation loss:  0.4346
        Last batch out of {3} has training loss: 0.3945
Epoch {42} Avg. training loss over all batches: 0.3739 Validation loss:  0.4343
        Last batch out of {3} has training loss: 0.3574
Epoch {43} Avg. training loss over all batches: 0.3675 Validation loss:  0.4285
        Last batch out of {3} has training loss: 0.3798
Epoch {44} Avg. training loss over all batches: 0.3834 Validation loss:  0.4268
        Last batch out of {3} has training loss: 0.3198
```

```
Epoch {45} Avg. training loss over all batches: 0.3790 Validation loss:  0.4243
        Last batch out of {3} has training loss: 0.3309
Epoch {46} Avg. training loss over all batches: 0.3631 Validation loss:  0.4240
        Last batch out of {3} has training loss: 0.3846
Epoch {47} Avg. training loss over all batches: 0.3616 Validation loss:  0.4183
        Last batch out of {3} has training loss: 0.3880
Epoch {48} Avg. training loss over all batches: 0.3501 Validation loss:  0.4165
        Last batch out of {3} has training loss: 0.4218
Epoch {49} Avg. training loss over all batches: 0.3772 Validation loss:  0.4132
        Last batch out of {3} has training loss: 0.3262
Epoch {50} Avg. training loss over all batches: 0.3608 Validation loss:  0.4130
        Last batch out of {3} has training loss: 0.3785
Epoch {51} Avg. training loss over all batches: 0.3469 Validation loss:  0.4074
        Last batch out of {3} has training loss: 0.4217
Epoch {52} Avg. training loss over all batches: 0.3557 Validation loss:  0.4052
        Last batch out of {3} has training loss: 0.3925
Epoch {53} Avg. training loss over all batches: 0.3649 Validation loss:  0.4086
        Last batch out of {3} has training loss: 0.3560
Epoch {54} Avg. training loss over all batches: 0.3536 Validation loss:  0.4119
        Last batch out of {3} has training loss: 0.3908
Epoch {55} Avg. training loss over all batches: 0.3726 Validation loss:  0.4108
        Last batch out of {3} has training loss: 0.3235
Epoch {56} Avg. training loss over all batches: 0.3622 Validation loss:  0.4058
        Last batch out of {3} has training loss: 0.3554
Epoch {57} Avg. training loss over all batches: 0.3399 Validation loss:  0.4054
        Last batch out of {3} has training loss: 0.4291
Epoch {58} Avg. training loss over all batches: 0.3615 Validation loss:  0.4037
        Last batch out of {3} has training loss: 0.3531
Epoch {59} Avg. training loss over all batches: 0.3514 Validation loss:  0.4013
        Last batch out of {3} has training loss: 0.3854
Epoch {60} Avg. training loss over all batches: 0.3695 Validation loss:  0.4041
        Last batch out of {3} has training loss: 0.3228
Epoch {61} Avg. training loss over all batches: 0.3614 Validation loss:  0.4060
        Last batch out of {3} has training loss: 0.3487
Epoch {62} Avg. training loss over all batches: 0.3391 Validation loss:  0.4092
        Last batch out of {3} has training loss: 0.4227
Epoch {63} Avg. training loss over all batches: 0.3517 Validation loss:  0.4089
        Last batch out of {3} has training loss: 0.3802
Epoch {64} Avg. training loss over all batches: 0.3378 Validation loss:  0.4070
        Last batch out of {3} has training loss: 0.4273
Epoch {65} Avg. training loss over all batches: 0.3387 Validation loss:  0.4040
        Last batch out of {3} has training loss: 0.4219
Epoch {66} Avg. training loss over all batches: 0.3471 Validation loss:  0.4041
        Last batch out of {3} has training loss: 0.3924
```

```
Epoch {67} Avg. training loss over all batches: 0.3578 Validation loss:  0.4046
        Last batch out of {3} has training loss: 0.3544
Epoch {68} Avg. training loss over all batches: 0.3495 Validation loss:  0.4041
        Last batch out of {3} has training loss: 0.3831
Epoch {69} Avg. training loss over all batches: 0.3688 Validation loss:  0.4036
        Last batch out of {3} has training loss: 0.3153
Epoch {70} Avg. training loss over all batches: 0.3570 Validation loss:  0.4034
        Last batch out of {3} has training loss: 0.3557
Epoch {71} Avg. training loss over all batches: 0.3498 Validation loss:  0.4033
        Last batch out of {3} has training loss: 0.3800
Epoch {72} Avg. training loss over all batches: 0.3477 Validation loss:  0.4039
        Last batch out of {3} has training loss: 0.3857
Epoch {73} Avg. training loss over all batches: 0.3282 Validation loss:  0.4034
        Last batch out of {3} has training loss: 0.4528
Epoch {74} Avg. training loss over all batches: 0.3558 Validation loss:  0.4031
        Last batch out of {3} has training loss: 0.3582
Epoch {75} Avg. training loss over all batches: 0.3566 Validation loss:  0.4026
        Last batch out of {3} has training loss: 0.3540
Epoch {76} Avg. training loss over all batches: 0.3575 Validation loss:  0.4015
        Last batch out of {3} has training loss: 0.3507
Epoch {77} Avg. training loss over all batches: 0.3479 Validation loss:  0.4014
        Last batch out of {3} has training loss: 0.3824
Epoch {78} Avg. training loss over all batches: 0.3572 Validation loss:  0.4015
        Last batch out of {3} has training loss: 0.3508
Epoch {79} Avg. training loss over all batches: 0.3673 Validation loss:  0.4010
        Last batch out of {3} has training loss: 0.3144
Epoch {80} Avg. training loss over all batches: 0.3559 Validation loss:  0.3998
        Last batch out of {3} has training loss: 0.3605
Epoch {81} Avg. training loss over all batches: 0.3667 Validation loss:  0.4019
        Last batch out of {3} has training loss: 0.3155
Epoch {82} Avg. training loss over all batches: 0.3456 Validation loss:  0.4030
        Last batch out of {3} has training loss: 0.3884
Epoch {83} Avg. training loss over all batches: 0.3373 Validation loss:  0.4032
        Last batch out of {3} has training loss: 0.4163
Epoch {84} Avg. training loss over all batches: 0.3655 Validation loss:  0.4006
        Last batch out of {3} has training loss: 0.3183
Epoch {85} Avg. training loss over all batches: 0.3528 Validation loss:  0.3977
        Last batch out of {3} has training loss: 0.3683
Epoch {86} Avg. training loss over all batches: 0.3523 Validation loss:  0.3985
        Last batch out of {3} has training loss: 0.3622
Epoch {87} Avg. training loss over all batches: 0.3519 Validation loss:  0.3984
        Last batch out of {3} has training loss: 0.3607
Epoch {88} Avg. training loss over all batches: 0.3592 Validation loss:  0.4074
        Last batch out of {3} has training loss: 0.3414
```

```
Epoch {89} Avg. training loss over all batches: 0.3447 Validation loss:  0.4069
        Last batch out of {3} has training loss: 0.3896
Epoch {90} Avg. training loss over all batches: 0.3423 Validation loss:  0.4018
        Last batch out of {3} has training loss: 0.3933
Epoch {91} Avg. training loss over all batches: 0.3419 Validation loss:  0.3985
        Last batch out of {3} has training loss: 0.3905
Epoch {92} Avg. training loss over all batches: 0.3523 Validation loss:  0.3994
        Last batch out of {3} has training loss: 0.3596
Epoch {93} Avg. training loss over all batches: 0.3404 Validation loss:  0.4098
        Last batch out of {3} has training loss: 0.3888
Epoch {94} Avg. training loss over all batches: 0.3426 Validation loss:  0.4046
        Last batch out of {3} has training loss: 0.3905
Epoch {95} Avg. training loss over all batches: 0.3631 Validation loss:  0.3944
        Last batch out of {3} has training loss: 0.3156
Epoch {96} Avg. training loss over all batches: 0.3521 Validation loss:  0.3999
        Last batch out of {3} has training loss: 0.3506
Epoch {97} Avg. training loss over all batches: 0.3514 Validation loss:  0.4033
        Last batch out of {3} has training loss: 0.3506
Epoch {98} Avg. training loss over all batches: 0.3610 Validation loss:  0.3970
        Last batch out of {3} has training loss: 0.3148
Epoch {99} Avg. training loss over all batches: 0.3499 Validation loss:  0.3991
        Last batch out of {3} has training loss: 0.3548
Epoch {100} Avg. training loss over all batches: 0.3578 Validation loss:  0.4018
        Last batch out of {3} has training loss: 0.3258
Epoch {150} Avg. training loss over all batches: 0.3562 Validation loss:  0.3896
        Last batch out of {3} has training loss: 0.3145
Epoch {200} Avg. training loss over all batches: 0.3556 Validation loss:  0.3875
        Last batch out of {3} has training loss: 0.3140
Epoch {250} Avg. training loss over all batches: 0.3242 Validation loss:  0.3868
        Last batch out of {3} has training loss: 0.4206
Epoch {300} Avg. training loss over all batches: 0.3447 Validation loss:  0.3864
        Last batch out of {3} has training loss: 0.3495
Epoch {350} Avg. training loss over all batches: 0.3552 Validation loss:  0.3860
        Last batch out of {3} has training loss: 0.3133
Epoch {400} Avg. training loss over all batches: 0.3551 Validation loss:  0.3858
        Last batch out of {3} has training loss: 0.3133
Epoch {450} Avg. training loss over all batches: 0.3551 Validation loss:  0.3857
        Last batch out of {3} has training loss: 0.3134
Epoch {500} Avg. training loss over all batches: 0.3550 Validation loss:  0.3856
        Last batch out of {3} has training loss: 0.3134
```

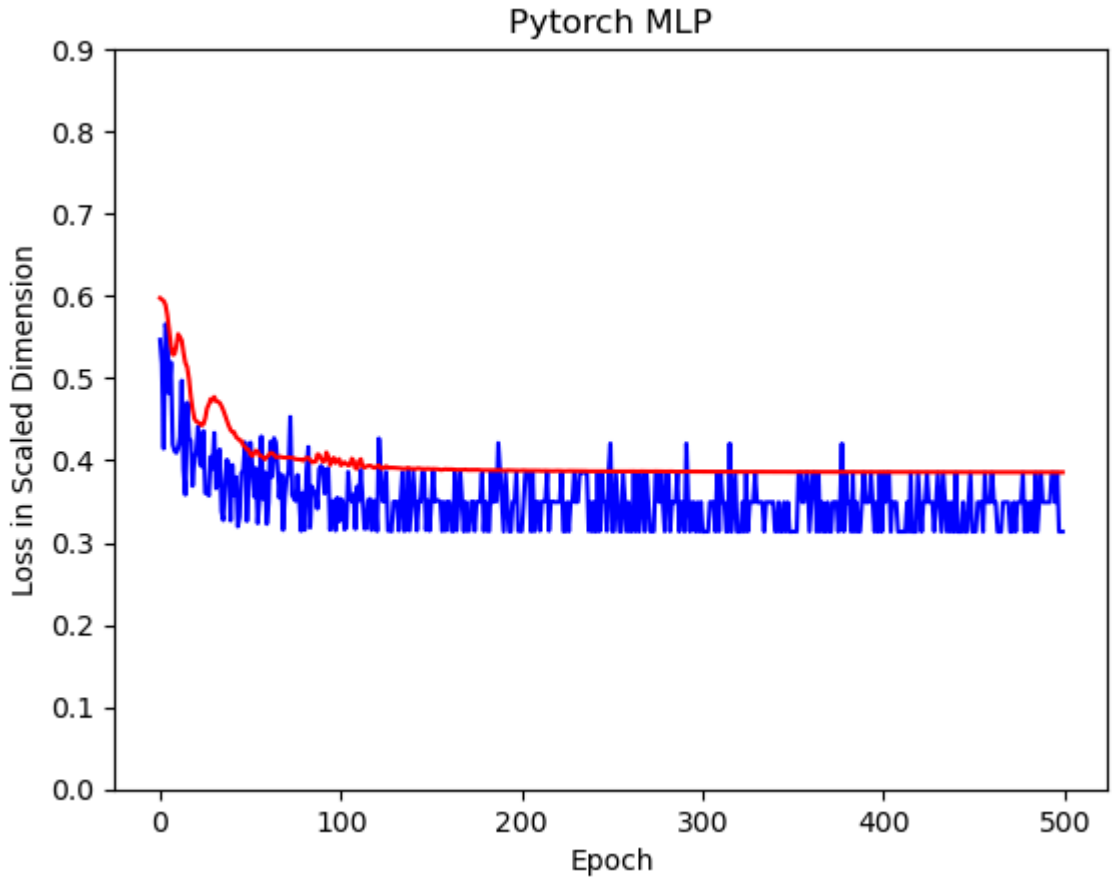
In [8]:
```
#inspect model
ig = IntegratedGradients(model_pt)#Specifically, integrated gradients
#are defined as the path intergral of the gradients along the straightline path
```

```
#see https://arxiv.org/pdf/1703.01365.pdf
ig_input_tensor = t_X_train
ig_input_tensor.requires_grad_()
attr, delta = ig.attribute(ig_input_tensor,target=0, return_convergence_delta=True)
#target class No 0 because we only have one output dimension
attr = attr.detach().numpy()
np.round(attr,2)
importances = np.mean(attr, axis=0)
feature_names = list(X_train.columns)
for i in range(len(feature_names)):
        print(feature_names[i], ": ", '%.3f'%(importances[i]))
```

```
SELBSTAENDIG :  -0.042
BERATUNGSOFFEN :  0.018
ALTER :  -0.011
MONATSNETTO :  -0.025
NOMINALZINS :  -0.083
GESAMTZUSAGE :  0.029
URSPRUNGSLAUFZEIT :  -0.041
RECHTVERFUEGBARSEIT :  -0.047
```

In [9]:

```
t_ctr = torch.arange(0, myMAX_EPOCHS, step = 1)
#t_ctr = t_ctr.clone().detach().unsqueeze(1)
plt.plot(t_ctr, losses_train, 'b') # plotting separately
plt.plot(t_ctr, losses_val, 'r')
plt.ylabel('Loss in Scaled Dimension')
plt.xlabel('Epoch')
plt.title('Pytorch MLP')
plt.ylim(top = 0.9)
plt.ylim(bottom = 0)
plt.show()
```



In [10]:
```
#forecast Logit
x_temp = sm.add_constant(X_train)
yhat_train = log_reg.predict(x_temp)
prediction_train = list(map(round, yhat_train))
x_temp = sm.add_constant(X_test)
yhat_test = log_reg.predict(x_temp)
prediction_test = list(map(round, yhat_test))
```

In [11]:
```
#forecasts MLP
out_probs_val = model_pt(t_X_valid).detach().numpy()
print(out_probs_val)
out_classes_val = np.argmax(out_probs_val, axis=1)
print(out_classes_val)
```

```
out_probs_train = model_pt(t_X_train).detach().numpy()
out_classes_train = np.argmax(out_probs_train, axis=1)

out_probs_test = model_pt(t_X_valid).detach().numpy()
out_classes_test = np.argmax(out_probs_test, axis=1)
```

```
[[2.3292138e-03 9.9767083e-01]
 [8.5959648e-04 9.9914038e-01]
 [1.0000000e+00 1.6446583e-21]
 [9.9999177e-01 8.2145398e-06]
 [9.9795222e-01 2.0477595e-03]
 [1.0000000e+00 2.9423741e-21]
 [1.7674839e-02 9.8232520e-01]
 [7.2046831e-05 9.9992800e-01]
 [9.9716043e-01 2.8395932e-03]
 [1.0000000e+00 3.4280630e-21]
 [1.0000000e+00 8.6655833e-22]
 [8.7544129e-07 9.9999917e-01]
 [9.9984765e-01 1.5238411e-04]
 [1.0000000e+00 2.2555141e-14]]
[1 1 0 0 0 0 1 1 0 0 0 1 0 0]
```

In [12]:

```
print("\nCOMPARISON ON TEST SET================")
print('Number of exercises in test data', sum(y_test))
print('As percentage %.3f ' %  (sum(y_test)/len(y_test)))
print('Benchmark Accuracy on train data %.3f '%  (1-sum(y_train)/len(y_train)))
print('Benchmark Accuracy on test data %.3f \n' %  (1-sum(y_test)/len(y_test)))

print('Accuracy  Logit on train data: %.3f ' % accuracy_score(y_train, prediction_train))
print('Confusion Matrix Logit on train data: \n', confusion_matrix(y_train, prediction_train))
print('Accuracy  Logit on test data: %.3f ' % accuracy_score(y_test, prediction_test))
#print('Precision Logit on test data: %.3f ' %  precision_score(y_test, prediction))
print('Confusion Matrix Logit on test data: \n', confusion_matrix(y_test, prediction_test))

#The precision is the ratio tp / (tp + fp) where tp is the number of true positives
#and fp the number of false positives. The precision is intuitively
#the ability of the classifier not to label as positive a sample that is negative.
#the best value is 1 and the worst value is 0.
#print('Benchmark Accuracy on test data %.3f \n' %  (1-sum(y_test)/len(y_test)))
print('Accuracy  MLP on train data: %.3f ' % (sum(out_classes_train == y_train) / len(y_train)))
print('Confusion Matrix MLP on train data:\n', confusion_matrix(y_train, out_classes_train))
print('Accuracy  MLP on test data: %.3f ' % (sum(out_classes_val == y_test) / len(y_test)))
print('Confusion Matrix MLP on test data:\n', confusion_matrix(y_test, out_classes_test))
print("\nXXX DONE XXX")
```

```
COMPARISON ON TEST SET=================
Number of exercises in test data 4.0
As percentage 0.286
Benchmark Accuracy on train data 0.782
Benchmark Accuracy on test data 0.714

Accuracy  Logit on train data: 0.847
Confusion Matrix Logit on train data:
 [[90  7]
 [12 15]]
Accuracy  Logit on test data: 0.643
Confusion Matrix Logit on test data:
 [[9 1]
 [4 0]]
Accuracy  MLP on train data: 0.968
Confusion Matrix MLP on train data:
 [[96  1]
 [ 3 24]]
Accuracy  MLP on test data: 0.929
Confusion Matrix MLP on test data:
 [[9 1]
 [0 4]]

XXX DONE XXX
```

In [ ]: